

ProductChat — How It Was Built

Step by step, from concept to a running, documented product.

01 The idea

- › Framed the problem: keyword search is brittle, shoppers don't know SKUs, generic AI hallucinates
- › Chose **RAG** — retrieve real products first; the model only explains, never invents

02 Data foundation

- › Scraped the live Metzler store (sitemap → JSON-LD product pages)
- › Extracted facts: name, SKU, price, image, category
- › Result: **842 products** across 6 categories

03 Storage design

- › Models: `Product`, `Settings`, `IndexingStatus`
- › Split on purpose: **relational DB** for facts, **Qdrant** for meaning (vectors)

04 Indexing pipeline

- › Composed each product into a text blob
- › Embedded with `mistral-embed` → **1024-dim vector**
- › Stored vectors in Qdrant (IDs mapped to stable UUIDs)

05 RAG engine

- Embed the question into the same vector space
- Cosine search in Qdrant → ~10 nearest products
- Send candidates to `mistral-large` (JSON: pick, explain, ask)
- **Guardrail in code**: drop any product ID the model invented

06 Supporting services

- **Suggestions**: free templates + cached LLM questions
- **Scheduler**: auto re-scrape, change detection (re-embed only what changed)
- **Settings**: DB-stored; API key **Fernet-encrypted**
- **Storefront proxy**: strips framing headers to embed the live shop

07 Interfaces

- **Backend**: FastAPI, endpoints under `/api`
- **Frontend**: React + Vite — Store, Chat, Explorer, Admin
- **Packaging**: `docker-compose` (Postgres + Qdrant + Redis + app)

THIS ENGAGEMENT

08 Making it run locally no Docker • no admin

- Diagnosed: **no Docker**; Node 22 + Python 3.14; Docker blocked (WSL2/Hyper-V need admin)
- Re-architected local: **SQLite** (not Postgres), **embedded Qdrant** (not the server), **no Redis**
- Swapped the Mistral SDK for **REST calls** — builds on Python 3.14 (old `orjson` wouldn't)
- Created `backend/.env`, a slim `requirements-local.txt`, and a one-click `run.bat`
- Built the venv, imported the 40 sample products, launched both servers, verified end-to-end

09 Documentation & materials

- System-review document (engineer / UX / AI lenses + step diagram + roadmap)
- Plain-language explanations of the mechanism & background
- **16-slide PowerPoint** presenting and defending the design, with a cost slide and a "how it was built" appendix
- Cost analysis on real data: ~\$0.02 to index the catalog, ~1¢ per chat

CURRENT STATE & HOW TO RUN

A working, locally-runnable RAG assistant with a review doc, a presentation with costs, and a phased roadmap (Phase 0 hardening → growth).

1. Set `MISTRAL_API_KEY` in `backend/.env` (or on the Settings page)
2. Double-click `run.bat`
3. Open **`http://localhost:3000`** · API docs at **`http://localhost:8000/docs`**
4. Run indexing from the Admin page to embed the products